
FlowGuard: Calibrated Success-Only Monitoring for Receding-Horizon Generative Robot Policies

Pei-Chi Chen

IIS, Academia Sinica, Taiwan
pcchen0413@iis.sinica.edu.tw

Tsung-Wei Ke

CSIE, National Taiwan University
twke@csie.ntu.edu.tw

Hwann-Tzong Chen

Dept. of CS, National Tsing Hua University
htchen@cs.nthu.edu.tw

Tyng-Luh Liu

IIS, Academia Sinica, Taiwan
liutyng@iis.sinica.edu.tw

Abstract

Failures in robotic manipulation are costly, diverse, and rarely available for training or calibration. We introduce FLOWGUARD, a success-only reliability layer for receding-horizon generative robot policies. FLOWGUARD respects the information timing of closed-loop control: before execution, an observation-conditioned VQ action tokenizer and causal plan-token success prior score sampled chunks using only admissible pre-execution information; after the selected prefix is executed, a feedback-dependent consistency model supplies post-execution failure evidence. Temporally correlated scores are normalized within progress bins, accumulated with a CUSUM-style statistic, and calibrated at the trajectory level using held-out successful rollouts. On RoboMimic, post-execution consistency is the strongest detection signal, while plan-likelihood and VQ-consistency are weak standalone detectors but useful pre-execution compatibility scores. In low-dimensional detection over three tasks, FLOWGUARD achieves macro AUROC 0.912 ± 0.09 and 6% mean FPR under success-only trajectory calibration, and is competitive but task-dependent against success-only baselines. Diagnostics further show that action-side failures must manifest in future observations, finite calibration budgets can create conservative threshold cliffs, and actor-matched calibration better controls false alarms under closed-loop shift. These results support timing-correct, success-only runtime monitoring for receding-horizon robot control.

1 Introduction

Generative robot policies increasingly cast control as conditional sequence generation. Recent methods sample multi-step action chunks and execute only a short prefix in a receding-horizon loop, enabling multimodal and coherent behavior in robotic manipulation [11, 42, 50]. However, this interface raises a reliability problem: generative policies may sample multiple plausible action chunks at each decision step, but the robot must choose among them before observing their consequences. Although a selected chunk may appear locally admissible before execution, contact uncertainty, slippage, perception error, and compounding control error can turn the executed prefix into a failed rollout [1, 38, 49].

Failure supervision is rarely available at deployment scale. Successful demonstrations are comparatively easy to collect, whereas failures are costly, heterogeneous, and tied to the task, environment, and deployed actor. We therefore study a *success-only* reliability setting: scoring models and alarm thresholds are learned or calibrated from successful rollouts, with failure labels and corruptions used only for evaluation. The central challenge is information timing. Before execution, a receding-horizon

controller can use only the current context, bounded plan history, masks, and sampled chunks. After execution, a monitor can use feedback observations to assess whether the realized transition remains compatible with success. Candidate ranking and failure monitoring may therefore share success-only data, but they must obey different admissible information sets.

We introduce FLOWGUARD, a success-only reliability layer for chunk-based generative robot policies. FLOWGUARD learns an observation-conditioned VQ action tokenizer, a causal plan-token success prior, and a post-execution consistency model from successful rollouts. Before execution, the tokenizer and prior produce timing-admissible compatibility scores: S_{plan} , which measures compatibility with recent successful plan-token history, and S_{vq} , which measures proximity to the learned success-token representation. These scores can rank sampled chunks without using future feedback. After the selected prefix is executed, FLOWGUARD computes S_{exec} , a feedback-dependent consistency score for monitoring. This separation is essential: the most informative failure evidence may be available only after execution and therefore cannot be used to select the action that generated it.

FLOWGUARD converts temporally correlated runtime scores into rollout-level alarms using success-only calibration. Rather than thresholding each cycle independently, it normalizes scores within task-specific progress bins, accumulates one-sided evidence with a drift-corrected CUSUM statistic [35], and sets a trajectory-level threshold from held-out successful rollouts using conformal calibration principles [2, 45]. The calibration target is the probability of at least one false alarm on a successful rollout. In closed loop, this target depends on the deployed actor: even successful policy-induced trajectories can shift away from demonstration statistics. Accordingly, when verified successful policy-induced rollouts are available, FLOWGUARD calibrates on rollouts matched to the actor, sampler, ranking rule, execution horizon, feature encoder, and scoring pipeline.

Experiments on RoboMimic support this timing-constrained view. The post-execution score S_{exec} is the dominant detection signal, whereas S_{plan} and S_{vq} are weak standalone detectors but valid pre-execution compatibility scores. Under shared success-only trajectory calibration, FLOWGUARD provides competitive, task-dependent failure detection rather than uniform dominance over generic novelty baselines. Additional diagnostics show that action-side failures must propagate into future observations to be detectable, finite calibration budgets can create conservative threshold cliffs, and actor-matched calibration is important for controlling false alarms under closed-loop shift. Thus, FLOWGUARD is best viewed as a calibrated post-execution monitor with timing-correct pre-execution compatibility scores, rather than a universal best-of- N policy-improvement mechanism.

Our contributions can be characterized as follows:

1. We formulate success-only reliability for receding-horizon generative robot policies as a timing-constrained problem that separates pre-execution compatibility scoring from post-execution failure detection.
2. We introduce FLOWGUARD, which combines a VQ action tokenizer, a causal plan-token success prior, and a feedback-dependent execution consistency model while enforcing admissible information use.
3. We develop success-only trajectory calibration with progress-binned normalization, one-sided CUSUM aggregation, and rollout-level conformal thresholding for false-alarm control on successful trajectories.
4. We show on RoboMimic that post-execution consistency is the primary detection signal, pre-execution ranking is actor-dependent, and actor-matched calibration is critical for closed-loop false-alarm control.

2 Related work

Chunk-based generative robot policies. Modern imitation policies increasingly generate action sequences rather than single-step commands. Behavior Transformers, ACT, and Diffusion Policy sample multi-step chunks for receding-horizon control [11, 42, 50], while flow matching and rectified flow provide continuous-time objectives for trajectory or action-chunk generation [26, 28]. This interface creates a runtime reliability problem beyond actor learning: chunks can be scored before execution, but the strongest evidence about the selected prefix may appear only after the robot has acted. FLOWGUARD addresses this timing constraint as a success-only reliability layer, not a new actor architecture. It separates pre-execution chunk compatibility from post-execution failure monitoring.

87 **Action tokenization and test-time scoring.** Discrete action abstractions provide compact structure
 88 for continuous robot behavior. VQ-style tokenization [43] has been used for behavior modeling,
 89 masked trajectory prediction, and scalable robot-action tokenization [36, 42, 47, 48]. Test-time
 90 methods also rank, filter, or guide generated trajectories using learned scores, density estimates,
 91 diffusion guidance, or self-evaluation signals [20, 29, 39, 44]. FLOWGUARD uses tokenization for a
 92 different role: the plan-token prior and VQ residual define scores available before execution, making
 93 them admissible for chunk compatibility checks. We do not assume these scores uniformly improve
 94 best-of- N closed-loop success; the feedback-dependent execution score is reserved for post-execution
 95 monitoring, where it provides the primary failure signal.

96 **Success-only runtime monitoring and failure detection.** Failure and out-of-distribution detection
 97 often use confidence scores, ensemble disagreement, reconstruction error, novelty estimation, or
 98 one-class objectives [10, 21, 24, 40]. In robotics, recent work studies success-only failure detection,
 99 progress monitoring, execution consistency, VLA-based failure reasoning, and runtime failure
 100 prediction for generative policies [1, 13, 17, 18, 38, 41, 49]. FLOWGUARD is closest to success-only
 101 runtime monitoring, but focuses on the information structure of receding-horizon execution. Pre-
 102 execution scores assess sampled chunks before acting, whereas transition evidence is used only after
 103 feedback is observed. This avoids leaking future feedback into action selection and keeps the monitor
 104 compatible with closed-loop deployment.

105 **Calibration for sequential alarms.** Calibration converts runtime scores into actionable alarms.
 106 Conformal prediction provides finite-sample calibration under exchangeability [2, 25, 45], with
 107 adaptive and non-exchangeable variants addressing distribution shift and alternative risk notions
 108 [3, 4, 14, 16]. Sequential monitoring adds a rollout-level complication: repeated per-cycle tests can
 109 inflate false alarms even when individual scores appear calibrated. Classical CUSUM detectors
 110 accumulate one-sided evidence over time [35], but their operating point must match the deployed
 111 score distribution. FLOWGUARD combines progress-binned normalization, CUSUM-style aggregation,
 112 and trajectory-level success-only calibration. When verified successful policy-induced rollouts are
 113 available, calibration is matched to the deployed actor, sampler, ranking rule, execution horizon,
 114 feature encoder, and scoring pipeline instead of relying only on demonstration statistics.

115 3 Problem setup

116 **Success-only data and task scope.** We describe a single task and omit the task index. In multi-task
 117 experiments, action normalizers, feature dimensions, progress bins, calibration statistics, and alarm
 118 thresholds are task-specific. Training contains only successful demonstrations $\mathcal{D}^+ = \{\tau_i\}_{i=1}^N$, where
 119 $\tau_i = (x_{1:T_i+1}, a_{1:T_i})$, x_t is the raw observation, and $a_t \in \mathbb{R}^{d_a}$ is applied before observing x_{t+1} . Let
 120 $\mathcal{N}_a$ be the task action normalizer, with $\tilde{a}_t = \mathcal{N}_a(a_t)$ and inverse $\mathcal{N}_a^{-1}$. No labeled failures are used
 121 to train the actor, tokenizer, success prior, execution model, score normalization, or alarm thresholds;
 122 failure labels and synthetic corruptions are used only for evaluation. When rollout-level success is
 123 verified, *e.g.*, in simulation or after completion, verified successful policy-induced rollouts $\mathcal{R}^{\text{pol},+}$
 124 may also be used for deployment-matched calibration; unverified rollouts are excluded.

125 **Feature states and deployment matching.** FLOWGUARD operates on deployed feature states
 126 $y_t = \phi_\omega(x_t)$. In low-dimensional experiments, x_t is the RoboMimic state and ϕ_ω is the identity map.
 127 In visual experiments, x_t contains images and proprioception, and ϕ_ω is a frozen visual-proprioceptive
 128 encoder. All modules and calibration quantities are tied to this feature map and to the deployed actor.
 129 Calibration is therefore recomputed whenever the actor, feature encoder, sampler, execution horizon,
 130 ranking rule, detection channels, or scoring pipeline changes.

131 **Chunked receding-horizon control.** At decision cycle j , the controller acts at time t_j , with $t_0 = 1$.
 132 Let H be the sampled chunk horizon and $H' \leq H$ the nominal execution horizon. Given a short
 133 context C_j , the actor samples N_{cand} normalized action chunks,

$$\tilde{A}_j^{(n)} = (\tilde{a}_{j,1}^{(n)}, \dots, \tilde{a}_{j,H}^{(n)}) \in \mathbb{R}^{H \times d_a}, \quad n = 1, \dots, N_{\text{cand}}. \quad (1)$$

134 By default, $C_j = (y_{\max\{1, t_j-1\}}, y_{t_j})$, with context mask $M_j^C = (\mathbf{1}\{t_j > 1\}, 1)$ for the padded first
 135 entry at $j = 0$. An action mask $M_j^A \in \{0, 1\}^H$ marks valid chunk positions near demonstration ends

or deployment-budget boundaries. A ranking rule selects candidate n_j^* . The controller executes only a prefix of length $H'_j \leq H'$. Under a finite control budget B_{ctrl} , $H'_j = \min\{H', B_{\text{ctrl}} - t_j + 1\}$; otherwise $H'_j = H'$ until task termination or alarm termination. The selected normalized prefix is mapped to environment actions as $A_j^{\text{env}} = \mathcal{N}_a^{-1}(\tilde{A}_{j,1:H'_j}^{(n_j^*)})$. After executing A_j^{env} , the monitor observes $y_{t_j+H'_j}$. If no terminal condition or alarm occurs, the next decision time is $t_{j+1} = t_j + H'_j$.

Bounded selected-token history. The online memory stores selected plan tokens rather than the full observation-action history. Let K_{code} be the tokenizer codebook size, and let $z_j^{\text{sel}} \in \{1, \dots, K_{\text{code}}\}$ be the token assigned to the selected horizon- H chunk under mask M_j^A . With a beginning-of-sequence token z_{BOS} and history length L_h , the bounded selected-token history before cycle j is

$$h_j = \text{pad}_{\text{BOS}}((z_i^{\text{sel}})_{i=\max\{0, j-L_h\}}^{j-1}) \in \{z_{\text{BOS}}, 1, \dots, K_{\text{code}}\}^{L_h}. \quad (2)$$

Thus h_j records recent intended plans in $O(L_h)$ tokens. Realized consequences are not stored in this history; they are assessed after execution from feedback observations.

Timing constraint. Receding-horizon control separates pre- and post-execution information. Before execution, candidate ranking may use only $\mathcal{I}_j^{\text{pre}} = (C_j, M_j^C, M_j^A, h_j, \{\tilde{A}_j^{(n)}\}_{n=1}^{N_{\text{cand}}})$, so deployable ranking is restricted to $\mathcal{Q}_{\text{rank}} = \{\text{plan}, \text{vq}\}$. After executing the selected prefix, the post-execution information set adds the selected index, executed length, and newly observed feedback state, $\mathcal{I}_j^{\text{post}} = (\mathcal{I}_j^{\text{pre}}, n_j^*, H'_j, y_{t_j+H'_j})$. Monitoring may then combine selected-chunk scores with feedback-dependent execution consistency, $\mathcal{Q}_{\text{det}} = \{\text{plan}, \text{vq}, \text{exec}\}$. Thus S_{exec} is valid for monitoring but invalid for ranking: its evidence exists only after the selected prefix has affected the world.

Trajectory-level calibration. Per-cycle scores are normalized by task progress. Let B_{mon} be the monitoring horizon, set to B_{ctrl} when a finite control budget is available and to a fixed nominal horizon otherwise. We define

$$\pi_j = \min \left\{ \frac{t_j - 1}{\max\{B_{\text{mon}} - 1, 1\}}, 1 \right\}, \quad (3)$$

and let b_j be the progress bin containing π_j . Score normalization, one-sided evidence aggregation, and alarm thresholds are estimated from successful rollouts only. The method maintains a trajectory statistic U_j , implemented as a CUSUM-style accumulation of calibrated one-sided evidence, and raises an alarm after cycle j when $U_{j+1} > h_{\text{traj}}$. The threshold h_{traj} targets a rollout-level false-alarm rate α_{FA} : the calibrated event is at least one false alarm on a successful trajectory, not an independent per-step error. When successful deployment traces shift away from demonstrations, verified successful policy-induced rollouts are preferred for calibration. Adaptive conformal updates, when used, update thresholds only from verified successful deployment rollouts.

Evaluation regimes and objective. We distinguish fixed-set evaluation from interventional closed-loop deployment. In fixed-set evaluation, score sets and calibration rules are replayed on the same traces, making AUROC and thresholded metrics directly comparable. In interventional closed-loop deployment, alarms truncate execution and change the observed rollout distribution; class counts are therefore rule-specific. For adaptive conformal inference, thresholds also evolve online. We therefore treat interventional closed-loop results as deployment diagnostics and report thresholded alarm statistics together with outcome counts.

Given successful demonstrations and, optionally, verified successful policy-induced calibration rollouts, our goal is a timing-correct reliability layer for chunked robot policies. Pre-execution plan and VQ scores provide admissible compatibility evidence for sampled chunks, while feedback-dependent execution consistency is reserved for post-execution monitoring. The primary objective is success-only, trajectory-level failure detection with calibrated false-alarm control. Pre-execution ranking is a secondary diagnostic and is not assumed to yield universal best-of- N policy improvement.

4 Method

FLOWGUARD is a success-only reliability layer for chunk-based generative robot policies. At each receding-horizon cycle, the actor samples candidate chunks, a timing-admissible rule selects one

before execution, only a prefix is executed, and monitoring updates after feedback is observed. FLOWGUARD preserves this order: pre-execution scores may rank candidates, whereas feedback-dependent consistency is used only for post-execution failure detection. All modules operate on the deployed feature states $y_t = \phi_\omega(x_t)$ from Section 3. The actor, sampler, feature encoder, tokenizer, prior, execution model, ranking rule, score definitions, detection channels, and channel weights are fixed before calibration; calibration estimates normalization statistics, reference levels, and trajectory thresholds for this fixed pipeline. Changing any component defines a new pipeline and requires recalibration.

4.1 Decision cycle and timing

At cycle j , the pre-execution information consists of context C_j , context mask M_j^C , action mask M_j^A , bounded selected-token history h_j , and sampled chunks $\{\tilde{A}_j^{(n)}\}_{n=1}^{N_{\text{cand}}}$, with $\tilde{A}_j^{(n)} \in \mathbb{R}^{H \times d_a}$. A ranking rule selects n_j^* using only this information. The controller executes the normalized prefix $\tilde{A}_{j,1:H_j'}^{(n_j^*)}$, maps it back to environment actions with $\mathcal{N}_a^{-1}$, and discards the suffix. Only after execution does the monitor observe $y_{t_j+H_j'}$. Scores depending on this feedback are post-execution quantities and are invalid for candidate ranking.

4.2 Success-token representation and causal prior

FLOWGUARD first learns a discrete representation of success-compatible action chunks. For a successful horizon- H chunk $(C_j, M_j^C, \tilde{A}_j, M_j^A)$, the tokenizer encoder produces $u_j = f_\eta(C_j, M_j^C, \tilde{A}_j, M_j^A) \in \mathbb{R}^{d_z}$. With codebook $\mathcal{E} = \{e_r\}_{r=1}^{K_{\text{code}}}$, the assigned token and code vector are

$$z_j = \arg \min_{r \in \{1, \dots, K_{\text{code}}\}} \|u_j - e_r\|_2^2, \quad q_j = e_{z_j}. \quad (4)$$

The tokenizer is trained on successful chunks using masked reconstruction, a VQ commitment term, and code-usage regularization; details are in Appendix A.2. For a sampled candidate $\tilde{A}_j^{(n)}$, let $u_j^{(n)}$, $z_j^{(n)}$, and $q_j^{(n)}$ be the corresponding latent, token, and code vector. Its VQ-consistency score is

$$S_{\text{vq}}(j, n) = \frac{1}{d_z} \|u_j^{(n)} - q_j^{(n)}\|_2^2. \quad (5)$$

This pre-execution score measures proximity to the learned success-token manifold.

The causal success prior models token sequences from successful rollouts. Let ϕ_C be the context encoder and $\gamma_j = \phi_C(C_j, M_j^C)$. From tokenized successful rollouts, we form tuples (C_j, M_j^C, h_j, z_j) , where h_j is the bounded history of preceding success tokens. The prior is trained by teacher forcing:

$$\mathcal{L}_{\text{prior}} = -\mathbb{E} \log p_\theta(z_j | h_j, \gamma_j). \quad (6)$$

During demonstration training, h_j contains previous demonstration tokens; during deployment and policy-induced calibration, it contains previously selected tokens. For candidate $\tilde{A}_j^{(n)}$, the plan-incompatibility score is

$$S_{\text{plan}}(j, n) = -\log p_\theta(z_j^{(n)} | h_j, \gamma_j). \quad (7)$$

Both S_{plan} and S_{vq} are timing-admissible compatibility scores. They can rank chunks before execution and provide auxiliary detection evidence after recalibration, but they are not assumed to be strong standalone failure detectors.

4.3 Pre-execution candidate ranking

Given sampled chunks, FLOWGUARD ranks candidates using only $\mathcal{Q}_{\text{rank}} = \{\text{plan}, \text{vq}\}$. The default rule is

$$n_j^* = \arg \min_n [\alpha_{\text{plan}} \bar{S}_{\text{plan}}^{\text{rank}}(j, n) + \alpha_{\text{vq}} \bar{S}_{\text{vq}}^{\text{rank}}(j, n)], \quad (8)$$

where $\alpha_{\text{plan}}, \alpha_{\text{vq}} \geq 0$ are fixed and not both zero. The barred terms are candidate scores normalized by a fixed ranking normalizer estimated before detector calibration. If ranking normalization is

218 disabled, we set $\bar{S}_\chi^{\text{rank}}(j, n) = S_\chi(j, n)$ for $\chi \in \mathcal{Q}_{\text{rank}}$. The selected token is $z_j^{\text{sel}} = z_j^{(n_j^*)}$, and
 219 the executed prefix is $\tilde{A}_j^+ = \tilde{A}_{j,1:H_j'}^{(n_j^*)}$. Equation (8) selects a candidate chunk rather than an alarm
 220 and excludes feedback-dependent scores. Empirically, ranking utility is actor-dependent, while the
 221 post-execution consistency score provides the strongest monitoring signal.

222 4.4 Post-execution consistency

223 After executing the selected prefix, FLOWGUARD observes $y_{t_j+H_j'}$. Let ϕ_o be the feedback encoder and
 224 $f_j^+ = \phi_o(y_{t_j+H_j'}) \in \mathbb{R}^{d_h}$. To keep a fixed horizon- H interface, the executed prefix $\tilde{A}_j^+$ is padded to
 225 $\tilde{A}_{j,\text{pad}}^+ \in \mathbb{R}^{H \times d_a}$ with mask $M_j^+ \in \{0, 1\}^H$. A diagonal Gaussian mixture execution model defines
 226 $p_\xi(f_j^+ \mid C_j, M_j^C, \tilde{A}_{j,\text{pad}}^+, M_j^+)$. The execution-consistency score is the dimension-normalized
 227 negative log-likelihood

$$S_{\text{exec}}(j) = -\frac{1}{d_h} \log p_\xi(f_j^+ \mid C_j, M_j^C, \tilde{A}_{j,\text{pad}}^+, M_j^+). \quad (9)$$

228 The execution model is trained only on successful transitions.

229 Unlike S_{plan} and S_{vq} , S_{exec} depends on post-execution feedback. It is used for monitoring and
 230 calibration, never for ranking. For the selected chunk, the default detection vector is

$$s_j = (S_{\text{plan}}(j, n_j^*), S_{\text{vq}}(j, n_j^*), S_{\text{exec}}(j)), \quad \mathcal{Q}_{\text{det}} = \{\text{plan}, \text{vq}, \text{exec}\}. \quad (10)$$

231 Changing $\mathcal{Q}_{\text{det}}$ or the channel weights defines a new detector and requires recalibration.

232 4.5 Success-only trajectory calibration

233 Calibration maps temporally correlated per-cycle scores into rollout-level alarms using only successful
 234 rollouts. Let $\mathcal{R}^{\text{cal},+}$ be the calibration set. Verified successful policy-induced rollouts are preferred
 235 when available because they match the deployed actor, sampler, ranking rule, execution horizon,
 236 feature encoder, and scoring pipeline. Otherwise, successful demonstrations serve as an offline proxy.

237 Calibration rollouts are split by rollout: $\mathcal{R}^{\text{cal},+} = \mathcal{R}^{\text{norm}} \cup \mathcal{R}^{\text{ref}} \cup \mathcal{R}^{\text{traj}}$. The split $\mathcal{R}^{\text{norm}}$ estimates
 238 score-normalization statistics, $\mathcal{R}^{\text{ref}}$ estimates CUSUM reference levels, and $\mathcal{R}^{\text{traj}}$ calibrates the
 239 trajectory-level alarm threshold. All hyperparameters are fixed before these calibration stages and are
 240 not tuned on any calibration split.

241 **Progress-binned evidence.** Scores are normalized within task-specific progress bins. For channel
 242 $\chi \in \mathcal{Q}_{\text{det}}$, let $(\hat{\mu}_{b,\chi}, \hat{\sigma}_{b,\chi})$ be shrinkage estimates from $\mathcal{R}^{\text{norm}}$, with pooled fallback for sparse
 243 bins and a minimum scale floor. Given score $s_{j,\chi}$ in bin b_j , define one-sided normalized evidence
 244 $\zeta_{j,\chi} = \max\{0, (s_{j,\chi} - \hat{\mu}_{b_j,\chi})/\hat{\sigma}_{b_j,\chi}\}$. The per-cycle aggregate is

$$r_j = \sum_{\chi \in \mathcal{Q}_{\text{det}}} w_\chi \zeta_{j,\chi}, \quad w_\chi \geq 0, \quad \sum_{\chi \in \mathcal{Q}_{\text{det}}} w_\chi = 1, \quad (11)$$

245 where channel weights are fixed before threshold calibration.

246 **CUSUM aggregation.** Reference levels are estimated from $\mathcal{R}^{\text{ref}}$. For each progress bin b , let κ_b be
 247 a fixed upper-tail reference level of r_j on successful reference cycles in that bin, with pooled fallback
 248 for sparse bins. With drift margin $\nu \geq 0$, online monitoring updates

$$U_0 = 0, \quad U_{j+1} = \max\{0, U_j + r_j - \kappa_{b_j} - \nu\}, \quad d_j = \mathbf{1}\{U_{j+1} > h_{\text{traj}}\}. \quad (12)$$

249 Here U_{j+1} is accumulated evidence after cycle j , and d_j is the corresponding alarm decision.

250 For each rollout in $\mathcal{R}^{\text{traj}}$, compute the trajectory maximum $V = \max_j U_{j+1}$. Let $V_{(1)} \leq \dots \leq$
 251 $V_{(M_{\text{traj}})}$ be the sorted maxima over $M_{\text{traj}} = |\mathcal{R}^{\text{traj}}|$ successful calibration rollouts. For a target
 252 trajectory-level false-alarm rate α_{FA} , define

$$\iota_{\text{traj}} = \lceil (M_{\text{traj}} + 1)(1 - \alpha_{\text{FA}}) \rceil. \quad (13)$$

253 If $\iota_{\text{traj}} \leq M_{\text{traj}}$, set $h_{\text{traj}} = V_{(\iota_{\text{traj}})}$; otherwise set $h_{\text{traj}} = +\infty$. Thus a finite threshold requires
 254 $\alpha_{\text{FA}} \geq 1/(M_{\text{traj}} + 1)$, equivalently $M_{\text{traj}} \geq \lceil 1/\alpha_{\text{FA}} \rceil - 1$. The strict alarm test in Eq. (12) gives
 255 conservative tie handling.

Proposition 1 (Trajectory-level false-alarm control). *Fix the deployed scoring pipeline, including the actor, sampler, feature encoder, tokenizer, prior, execution model, ranking rule, score definitions, detection channels, channel weights, score normalizers, reference levels, and trajectory-thresholding rule. If the calibration maxima in $\mathcal{R}^{\text{traj}}$ and the maximum V_{new} of a future successful rollout are exchangeable, then the static split-conformal threshold satisfies*

$$\Pr[V_{\text{new}} > h_{\text{traj}}] \leq \alpha_{\text{FA}}. \quad (14)$$

Equivalently, the successful-rollout false-alarm probability is at most α_{FA} .

The proposition follows from the exchangeable rank of V_{new} among the $M_{\text{traj}} + 1$ trajectory maxima. The strict alarm test makes ties conservative, and $h_{\text{traj}} = +\infty$ yields zero alarms when the conformal index exceeds the calibration set. The guarantee applies to the calibrated successful rollout distribution, but not directly to rule-dependent closed-loop rollouts truncated by alarms.

Adaptive deployment calibration. When successful deployment rollouts are sequentially verified, we optionally wrap the static threshold with adaptive conformal updates. The wrapper uses only verified successful rollouts to adjust the threshold toward α_{FA} ; it never uses failed rollouts or failure labels. Because thresholds change online and alarms can truncate execution, adaptive deployment results report thresholded alarm statistics and rule-specific outcome counts rather than AUROC.

5 Experiments

We study FLOWGUARD as a success-only reliability layer for receding-horizon robot policies through four questions: (i) whether post-execution consistency provides the strongest calibrated failure signal; (ii) whether plan and VQ scores provide timing-admissible pre-execution compatibility; (iii) how failure construction and calibration affect false alarms; and (iv) how visual features, actor quality, and actor-induced shift affect closed-loop deployment.

Protocol. We evaluate five RoboMimic tasks: LIFT, CAN, SQUARE, TOOLHANG, and TRANSPORT. Unless stated otherwise, each task uses 120 successful demonstrations split into 60 training, 40 calibration, and 20 evaluation trajectories. The 40 calibration trajectories are further split 16:12:12 for score normalization, reference-level estimation, and trajectory-threshold calibration. Hyperparameters are fixed a priori. Low-dimensional experiments use RoboMimic state features. Visual experiments use frozen features $y_t = \phi_\omega(x_t)$ on LIFT, CAN, and SQUARE. Unless noted, modules are trained per task with $K_{\text{code}} = 64$, $d_z = 64$, hidden width 128, two transformer layers, $H = 8$, $H' = 4$, and $L_h = 16$. Candidate ranking uses $\mathcal{Q}_{\text{rank}} = \{\text{plan}, \text{vq}\}$, whereas detection uses $\mathcal{Q}_{\text{det}} = \{\text{plan}, \text{vq}, \text{exec}\}$ with target false-alarm rate $\alpha_{\text{FA}} = 0.10$. The feedback-dependent score S_{exec} is never used for ranking.

Metrics. For fixed scores on fixed rollout sets, we report AUROC, balanced accuracy, TPR, and FPR, with FPR measured on successful rollouts. For adaptive conformal inference (ACI), thresholds update online from verified successful rollouts. In interventional closed-loop deployment, alarms can truncate execution and shift the rollout distribution; we therefore omit AUROC and report thresholded alarm statistics with rule-specific counts. Fixed-set closed-loop replays are reported separately, applying calibration rules counterfactually to the same untruncated traces.

5.1 Post-execution detection

Under the shared success-only calibration protocol, Table 1 separates the three score channels: S_{exec} is the dominant detection signal, while S_{plan} and S_{vq} are weak standalone detectors that add limited evidence after recalibration. This supports the timing split: plan and VQ scores are available before acting, whereas the strongest failure evidence appears only after the selected prefix affects the world. Table 2 compares FLOWGUARD with representative success-only baselines—random-network novelty [10], reconstruction, one-class objectives [40], ensemble uncertainty [24], forward-dynamics residuals, and progress monitoring—under the same splits, progress-binned normalization, CUSUM aggregation, and trajectory-level thresholding. No method dominates: forward dynamics is strongest on LIFT, one-class methods are competitive on SQUARE, and FLOWGUARD is strongest on CAN. Thus, FLOWGUARD is competitive but task-dependent, with its main signal from plan-conditioned post-execution consistency rather than generic novelty alone; the final two rows isolate S_{exec} -only and full FLOWGUARD, which adds plan and VQ evidence after recalibration.

Five-task coverage. On all five low-dimensional tasks, the default detector reaches macro AUROC 0.927 ± 0.029 and macro balanced accuracy 0.806 ± 0.083 over five seeds; full per-task results are

Table 1: Score-channel ablation for low-dimensional post-execution detection, macro-averaged over LIFT, CAN, and SQUARE. Each score set is recalibrated using successful rollouts only.

Score set	BalAcc	AUROC	TPR	FPR
S_{plan} only	0.501 $\pm$ 0.06	0.443 $\pm$ 0.16	48 $\pm$ 36%	50 $\pm$ 46%
S_{vq} only	0.497 $\pm$ 0.01	0.432 $\pm$ 0.03	6 $\pm$ 11%	7 $\pm$ 11%
$S_{\text{plan}} + S_{\text{vq}}$	0.514 $\pm$ 0.04	0.468 $\pm$ 0.09	26 $\pm$ 31%	24 $\pm$ 34%
S_{exec} only	0.756$\pm$0.09	0.901 $\pm$ 0.13	60$\pm$17%	8 $\pm$ 12%
FLOWGUARD	0.748 $\pm$ 0.08	0.912$\pm$0.09	58 $\pm$ 16%	6$\pm$10%

Table 2: Same-setting success-only baseline comparison on the three-task subset. Values are AUROC mean $\pm$ std over five seeds.

Method	LIFT	CAN	SQUARE
RND	0.876 $\pm$ 0.129	0.805 $\pm$ 0.116	0.830 $\pm$ 0.058
Forward dynamics residual	0.985$\pm$0.014	0.670 $\pm$ 0.033	0.688 $\pm$ 0.031
Autoencoder reconstruction	0.935 $\pm$ 0.110	0.833 $\pm$ 0.088	0.873 $\pm$ 0.066
Deep SVDD	0.959 $\pm$ 0.055	0.865 $\pm$ 0.158	0.932 $\pm$ 0.040
Ensemble uncertainty	0.652 $\pm$ 0.116	0.635 $\pm$ 0.052	0.673 $\pm$ 0.057
Progress monitor	0.849 $\pm$ 0.072	0.794 $\pm$ 0.165	0.897 $\pm$ 0.036
S_{exec} only (FLOWGUARD variant)	0.797 $\pm$ 0.174	0.921 $\pm$ 0.054	0.970$\pm$0.022
FLOWGUARD	0.853 $\pm$ 0.132	0.927$\pm$0.044	0.962 $\pm$ 0.037

in Appendix B.1. AUROC is high on CAN (0.980 $\pm$ 0.018), SQUARE (0.949 $\pm$ 0.024), TOOLHANG (0.993 $\pm$ 0.010), and TRANSPORT (0.928 $\pm$ 0.029), but lower on LIFT (0.783 $\pm$ 0.062). Thresholded metrics are more calibration-sensitive: TRANSPORT has FPR 47 $\pm$ 41%, consistent with unstable successful-score upper tails. Thus, the five-task result supports score separability, while the closed-loop study below focuses on false-alarm control under actor-induced shift. Appendix B.9 further reports score-weight robustness over all five tasks.

5.2 Failure construction and calibration

Action-side corruptions must affect future observations: pairing corrupted actions with the original successful observation sequence leaves feedback success-like and artificially weakens S_{exec} . Appendix B.2 shows that non-reexecuted corruptions understate detectability, whereas simulator re-execution or residual-dynamics re-simulation yields self-consistent action-side failures; we therefore use propagated action-side failures by default. Calibration can also fail despite score separability when the success-only trajectory threshold is overly conservative. Figure 1 shows this finite-sample cliff on LIFT: with too few successful threshold-calibration rollouts, split conformal sets $h_{\text{traj}} = \infty$, suppressing all alarms. Conformal risk control (CRC) and linearly interpolated quantiles provide finite-threshold operating-point diagnostics, while split conformal remains the default once sufficient successful calibration rollouts are available. Additional diagnostics appear in Appendices B.2 and B.3.

Calibration-design ablation. Appendix B.7 studies rollout-level calibration on LIFT, CAN, and SQUARE. All calibrated trajectory-level variants keep FPR near nominal (0.05–0.08). By contrast, a per-cycle diagnostic without trajectory-level thresholding over-fires severely (0.894 $\pm$ 0.094 FPR), since repeated cycle tests inflate rollout-level alarm probability. The full design—progress bins, CUSUM aggregation, per-bin reference levels, and a trajectory-level threshold—achieves the lowest FPR (0.050 $\pm$ 0.045) at moderate TPR, supporting rollout-level rather than per-step calibration.

5.3 Pre-execution candidate selection

We next test whether pre-execution scores can rank candidate chunks before feedback is available. This is a timing-admissibility diagnostic, not a claim of universal best-of- N policy improvement; S_{exec} is excluded because it depends on post-execution feedback. Appendix B.4 reports two diagnostics. In low-dimensional smoothness tests, $S_{\text{plan}} + S_{\text{vq}}$ selects more coherent chunks than random sampling, supporting their use as pre-execution compatibility signals. However, with Flow-DP-Distill, a DP-teacher-distilled flow actor detailed in Appendix A.3, closed-loop success may not improve and can underperform random selection. We therefore use these pre-execution scores as timing-correct compatibility evidence, not as guaranteed success-improving selectors. Their rollout-level utility is actor-dependent, whereas calibrated monitoring relies on post-execution evidence.

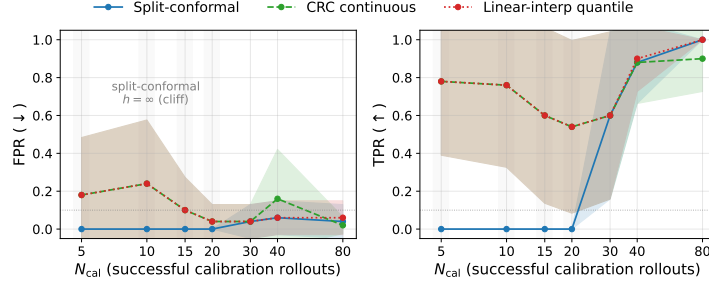


Figure 1: **Calibration-budget sensitivity on LIFT.** The shaded region marks the split-conformal finite-sample cliff where $h_{\text{traj}} = \infty$. Above the cliff, split conformal keeps FPR near $\alpha_{\text{FA}} = 0.10$; CRC and linearly interpolated quantiles provide finite-threshold diagnostics.

340 5.4 Visual monitoring and actor-induced calibration

341 Visual monitoring introduces two shifts: the frozen feature map ϕ_ω may not preserve task-relevant
 342 state geometry, and the deployed actor may produce successful trajectories whose score statistics differ
 343 from demonstrations. Appendix B.5 reports offline frozen-encoder diagnostics: same-task DP-init
 344 features are strongest, while generic ImageNet features are weaker, suggesting that task-aligned visual
 345 representations are important for success-only monitoring. Appendix B.9 additionally shows that
 346 feature-space nuisance proxies are pessimistic relative to engine-side state jitter.

347 **Actor quality and monitor intervention.** Closed-loop monitoring is interventional: alarms truncate
 348 execution and alter the rollout distribution induced by each calibration rule. We therefore use three
 349 diagnostics. The no-monitor baseline disables alarm termination and measures actor competence
 350 while saving score traces. Interventional deployment regenerates rollouts under each calibration rule
 351 and reports rule-specific successes, failures, and alarm aborts. Fixed-set replay applies calibration
 352 rules to the same no-monitor traces, isolating calibration shift from monitor-induced truncation.

353 Table 10 summarizes visual closed-loop deployment over three seeds and eight rollouts per seed. No-
 354 monitor rollouts separate actor competence from monitor behavior. LIFT is the clearest alarm-control
 355 setting: without alarm termination, the actor succeeds in 22/24 rollouts with BC-RNN features and
 356 21/24 with DP-init features, whereas demo-static calibration aborts all 24 rollouts for both backbones.
 357 Actor-induced calibration with ACI reduces alarm aborts to 3/24 and yields verified-success alarm
 358 ratios of 1/18 and 3/22, near $\alpha_{\text{FA}} = 0.10$. On CAN, the effect is representation-dependent: ACI
 359 improves completed successes and reduces aborts with BC-RNN features, while DP-init features
 360 remain mixed. SQUARE is actor-bound, with only 2/24 and 1/24 no-monitor successes, so its
 361 closed-loop statistics are low-success diagnostics rather than evidence against calibration. Full counts,
 362 no-monitor/fixed-set replays, and caught/missed rollouts are in Appendices B.6, B.8, and B.10.

363 **Fixed-set calibration replay.** Interventional counts mix calibration quality with rule-specific
 364 truncation, so we replay all rules on the same no-monitor score traces. On LIFT, demo-static
 365 calibration gives FAR = 1.00 for both visual backbones, whereas actor-induced static calibration
 366 reduces FAR to 0.00. On CAN, demo-static again gives FAR = 1.00 for both backbones; actor-induced
 367 static calibration reduces FAR to 0.00 for DP-init features but remains 0.57 for BC-RNN features.
 368 Thus, demo-static over-firing reflects genuine actor-induced calibration shift, not rule-specific rollout
 369 truncation. The SQUARE cells are noisy because the actor yields few verified successful traces.

370 **Closed-loop interpretation.** The closed-loop study supports deployment calibration, not universal
 371 success improvement. Actor-matched calibration can reduce severe false-alarm overfiring when
 372 successful actor-induced rollouts depart from demonstration statistics, but the gains depend on actor
 373 competence and representation quality. No-monitor rollouts test whether the actor yields enough
 374 successful traces for meaningful evaluation; interventional counts measure deployment cost; fixed-set
 375 replay verifies that FPR changes come from calibration rather than changed rollout sets.

376 6 Conclusion

377 We introduced FLOWGUARD, a success-only reliability layer for receding-horizon generative robot
 378 policies. It learns a discrete success-token representation, a causal plan-token prior, and a post-
 379 execution consistency model while preserving deployment timing: sampled chunks are ranked only
 380 with pre-execution plan and VQ compatibility scores, whereas feedback-dependent consistency is
 381 used only after executing the selected prefix. Across RoboMimic tasks, FLOWGUARD provides

competitive, task-dependent success-only failure detection under trajectory-level calibration. S_{exec} is the dominant detection signal, while S_{plan} and S_{vq} are weak standalone detectors but valid pre-execution compatibility scores; best-of- N gains are therefore actor-dependent rather than universal. Results also highlight practical deployment requirements: action-side failures should propagate into future observations, calibration needs enough successful rollouts to avoid finite-sample threshold cliffs, and false-alarm control benefits from deployment-matched successful rollouts under actor-induced shift. **Limitations** include no per-step false-alarm guarantee, reliance on successful deployment calibration traces, possible missed failures under weak feedback, and nuisance sensitivity from frozen visual encoders. Future work includes stronger visual feedback models, longer-horizon success priors, calibrated recovery, and adaptive calibration from verified successful deployment rollouts.

References

- [1] Christopher Agia, Rohan Sinha, Jingyun Yang, Ziang Cao, Rika Antonova, Marco Pavone, and Jeannette Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *CoRL*, 2024.
- [2] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *Found. Trends Mach. Learn.*, 2023.
- [3] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *ICLR*, 2024.
- [4] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. Conformal prediction beyond exchangeability. *Ann. Stat.*, 2023.
- [5] Johan Bjorck et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [6] Kevin Black et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [7] Kevin Black et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. In *CoRL*, 2025.
- [8] Anthony Brohan et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2023.
- [9] Anthony Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *CoRL*, 2023.
- [10] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *ICLR*, 2019.
- [11] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *IJRR*, 2024.
- [12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*, 2019.
- [13] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language model for detecting and reasoning over failures in robotic manipulation. In *ICLR*, 2025.
- [14] António Farinhas, Chrysoula Zerva, Dennis Ulmer, and André F. T. Martins. Non-exchangeable conformal risk control. In *ICLR*, 2024.
- [15] Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, et al. Gemini robotics: Bringing AI into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [16] Isaac Gibbs and Emmanuel J. Candès. Adaptive conformal inference under distribution shift. In *NeurIPS*, 2021.
- [17] Clemence Grislain, Hamed Rahimi, Olivier Sigaud, and Mohamed Chetouani. I-FailSense: Towards general robotic failure detection with vision-language models. In *ICRA*, 2026.
- [18] Qiao Gu, Yuanliang Ju, Shengxiang Sun, Igor Gilitschenski, Haruki Nishimura, Masha Itkina, and Florian Shkurti. SAFE: Multitask failure detection for vision-language-action models. In *NeurIPS*, 2025.

- 430 [19] Chengyang He, Xu Liu, Gadiel Mark Sznajder Camps, Joseph Bruno, Guillaume Adrien Sartoretti, and
431 Mac Schwager. Demystifying robot diffusion policies: Action memorization and a simple lookup table
432 alternative. In *ICLR*, 2026.
- 433 [20] Nantian He, Shaohui Li, Zhi Li, Yu Liu, and You He. ReDiffuser: Reliable decision-making using a diffuser
434 with confidence estimation. In *ICML*, 2024.
- 435 [21] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples
436 in neural networks. In *ICLR*, 2017.
- 437 [22] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation
438 dataset. In *RSS*, 2024.
- 439 [23] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint*
440 *arXiv:2406.09246*, 2024.
- 441 [24] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive
442 uncertainty estimation using deep ensembles. In *NeurIPS*, 2017.
- 443 [25] Jing Lei, Max G’Sell, Alessandro Rinaldo, Ryan J. Tibshirani, and Larry Wasserman. Distribution-free
444 predictive inference for regression. *JASA*, 2018.
- 445 [26] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching
446 for generative modeling. In *ICLR*, 2023.
- 447 [27] Bo Liu et al. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets*
448 *and Benchmarks*, 2023.
- 449 [28] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer
450 data with rectified flow. In *ICLR*, 2023.
- 451 [29] Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. Bidirectional
452 decoding: Improving action chunking via guided test-time sampling. In *ICLR*, 2025.
- 453 [30] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Fei-Fei Li,
454 Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human
455 demonstrations for robot manipulation. In *CoRL*, 2021.
- 456 [31] Ajay Mandlekar et al. MimicGen: A data generation system for scalable robot learning using human
457 demonstrations. In *CoRL*, 2023.
- 458 [32] Alex Mitrevski and Ayush Salunke. COFFAIL: A dataset of successful and anomalous robot skill executions
459 in the context of coffee preparation. *arXiv preprint arXiv:2604.18236*, 2026.
- 460 [33] Octo Model Team et al. Octo: An open-source generalist robot policy. In *RSS*, 2024.
- 461 [34] Open X-Embodiment Collaboration et al. Open X-embodiment: Robotic learning datasets and RT-X
462 models. In *ICRA*, 2024.
- 463 [35] Ewan S Page. Continuous inspection schemes. *Biometrika*, 1954.
- 464 [36] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea
465 Finn, and Sergey Levine. FAST: Efficient action tokenization for vision-language-action models. *arXiv*
466 *preprint arXiv:2501.09747*, 2025.
- 467 [37] Scott Reed et al. A generalist agent. *TMLR*, 2022.
- 468 [38] Ralf Römer, Adrian Kobras, Luca Worbis, and Angela P Schoellig. Failure prediction at runtime for
469 generative robot policies. In *NeurIPS*, 2025.
- 470 [39] Andrea Rosasco, Federico Ceola, Giulia Pasquale, and Lorenzo Natale. KDPE: A kernel density estimation
471 strategy for diffusion policy trajectory selection. *CoRL*, 2025.
- 472 [40] Lukas Ruff, Robert A. Vandermeulen, Nico Görnitz, Lucas Deecke, Shoaib A. Siddiqui, Alexander Binder,
473 Emmanuel Müller, and Marius Kloft. Deep one-class classification. In *ICML*, 2018.
- 474 [41] Som Sagar, Jiafei Duan, Sreevishakh Vasudevan, Yifan Zhou, Heni Ben Amor, Dieter Fox, and Ransalu
475 Senanayake. RoboMD: Uncovering robot vulnerabilities through semantic potential fields. In *ICLR*, 2026.

- 476 [42] Nur Muhammad Mahi Shafiullah, Zichen Jeff Cui, Ariuntuya Altanzaya, and Lerrel Pinto. Behavior
477 transformers: Cloning k modes with one stone. In *NeurIPS*, 2022.
- 478 [43] Aäron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In
479 *NeurIPS*, 2017.
- 480 [44] Marcel Torne Villasevil, Andy Tang, Yuejiang Liu, and Chelsea Finn. Learning long-context diffusion
481 policies via past-token prediction. In *CoRL*, 2025.
- 482 [45] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*.
483 Springer, 2005.
- 484 [46] Homer Rich Walke et al. BridgeData V2: A dataset for robot learning at scale. In *CoRL*, 2023.
- 485 [47] Yating Wang, Haoyi Zhu, Mingyu Liu, Jiange Yang, Hao-Shu Fang, and Tong He. VQ-VLA: Improving
486 vision-language-action models via scaling vector-quantized action tokenizers. In *ICCV*, 2025.
- 487 [48] Philipp Wu, Arjun Majumdar, Kevin Stone, Yixin Lin, Igor Mordatch, Pieter Abbeel, and Aravind
488 Rajeswaran. Masked trajectory models for prediction, representation, and control. In *ICML*, 2023.
- 489 [49] Chen Xu, Tony Khuong Nguyen, Emma Dixon, Christopher Rodriguez, Patrick Miller, Robert Lee, Paarth
490 Shah, Rares Ambrus, Haruki Nishimura, and Masha Itkina. Can we detect failures without failure data?
491 Uncertainty-aware runtime failure detection for imitation learning policies. In *RSS*, 2025.
- 492 [50] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual
493 manipulation with low-cost hardware. In *RSS*, 2023.

A Additional method details

This appendix gives implementation details deferred from Section 4: online deployment, tokenizer training, actor instantiation, ranking normalization, execution-consistency modeling, and success-only calibration details. Unless stated otherwise, we describe one task and omit the task index. The actor, sampler, feature encoder, tokenizer, prior, execution model, ranking rule, score definitions, detection channels, and channel weights are fixed before calibration; calibration estimates normalization statistics, reference levels, and trajectory thresholds for this fixed pipeline. Any pipeline change requires recalibration.

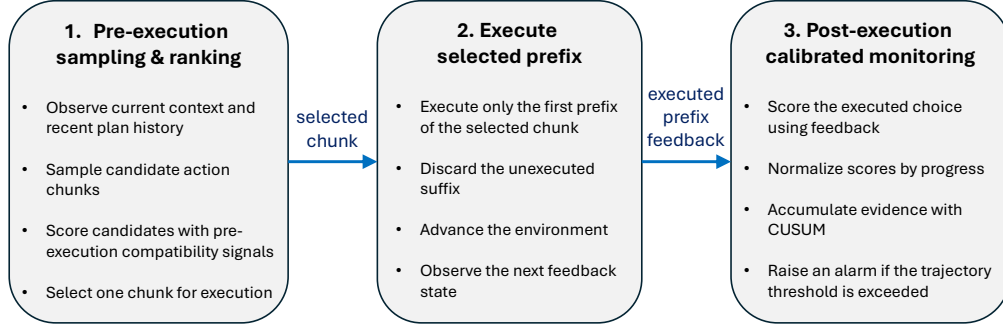


Figure 2: **FLOWGUARD decision cycle.** One receding-horizon decision has three stages: pre-execution sampling and ranking, prefix execution, and post-execution monitoring. The monitor updates a calibrated trajectory-level statistic only after feedback is observed, preventing feedback-dependent failure evidence from leaking into action selection.

A.1 Online FLOWGUARD deployment

Figure 2 illustrates one decision cycle. At cycle j , the actor receives C_j , masks M_j^C, M_j^A , and bounded selected-token history h_j , then samples normalized horizon- H chunks $\{\tilde{A}_j^{(n)}\}_{n=1}^{N_{\text{cand}}}$. Before execution, FLOWGUARD may use only the context, masks, history, and sampled chunks, so candidate ranking uses only S_{plan} and S_{vq} . The selected chunk determines $z_j^{\text{sel}} = z_j^{(n_j^*)}$, but only a prefix of length $H'_j \leq H'$ is executed before replanning. After execution, the robot observes $y_{t_j+H'_j}$, enabling S_{exec} for monitoring and calibration. Since S_{exec} depends on future feedback, it is never used for candidate ranking.

Algorithm 1 makes the timing constraint explicit: pre-execution scores may affect action selection, whereas S_{exec} is computed only after feedback is observed.

A.2 Tokenizer and codebook maintenance

Chunk construction and masking. For each successful demonstration, we form decision indices t_j according to the receding-horizon schedule and extract normalized horizon- H chunks $\tilde{A}_j$. Chunks extending beyond the demonstration or control budget are padded, with invalid entries masked by M_j^A . The action normalizer $\mathcal{N}_a$ is estimated from successful demonstrations and then fixed. All masked losses are normalized by the number of valid action entries; horizon masks are broadcast over the d_a action coordinates.

Tokenizer loss. The decoder g_ψ reconstructs the normalized chunk from the context, code vector, and mask. The tokenizer is trained only on successful chunks:

$$\mathcal{L}_{\text{tok}} = \frac{\left\| M_j^A \odot (g_\psi(C_j, M_j^C, q_j, M_j^A) - \tilde{A}_j) \right\|_2^2}{d_a \max\{\sum_{h=1}^H M_{j,h}^A, 1\}} + \beta_{\text{vq}} \|u_j - \text{sg}(q_j)\|_2^2 + \lambda_{\text{usage}} \mathcal{L}_{\text{usage}}. \quad (15)$$

Here $\text{sg}(\cdot)$ denotes stop-gradient, $\mathcal{L}_{\text{usage}} = \sum_{r=1}^{K_{\text{code}}} \bar{p}_r \log(\bar{p}_r + \epsilon_{\text{usage}})$, and $\bar{p}_r$ is the minibatch assignment frequency of code r . The nearest-code assignment is defined in Eq. (4).

Algorithm 1: FLOWGUARD deployment with bounded selected-token history

Input: Chunk sampler G_φ ; tokenizer $(f_\eta, \mathcal{E}, g_\psi)$; prior p_θ ; execution model p_ξ ; frozen ranking rule; calibrated score normalizers, reference levels, and threshold h_{traj} ; horizons H, H' ; candidate count N_{cand} ; history length L_h ; action normalizer $\mathcal{N}_a$; optional control budget B_{ctrl}

Output: Executed actions and, if detected, an alarm time

```
1 Initialize  $j \leftarrow 0, t_0 \leftarrow 1, U_0 \leftarrow 0$ , and  $h_0 \leftarrow (z_{\text{BOS}}, \dots, z_{\text{BOS}})$ 
2 while the task is active do
3   Form  $C_j, M_j^C$ , and  $M_j^A$ ; set  $\gamma_j \leftarrow \phi_C(C_j, M_j^C)$ 
4   Sample normalized chunks  $\{\tilde{A}_j^{(n)}\}_{n=1}^{N_{\text{cand}}} \sim G_\varphi(\cdot \mid C_j, M_j^C, M_j^A, h_j)$ 
5   for  $n = 1, \dots, N_{\text{cand}}$  do
6     Tokenize  $\tilde{A}_j^{(n)}$  to obtain  $u_j^{(n)}, z_j^{(n)}, q_j^{(n)}$ 
7     Compute  $S_{\text{vq}}(j, n) = d_z^{-1} \|u_j^{(n)} - q_j^{(n)}\|_2^2$ 
8     Compute  $S_{\text{plan}}(j, n) = -\log p_\theta(z_j^{(n)} \mid h_j, \gamma_j)$ 
9     Form  $S_{\text{rank}}(j, n) = \alpha_{\text{plan}} \tilde{S}_{\text{plan}}^{\text{rank}}(j, n) + \alpha_{\text{vq}} \tilde{S}_{\text{vq}}^{\text{rank}}(j, n)$ 
10    Select  $n_j^* \leftarrow \arg \min_n S_{\text{rank}}(j, n)$ , and set  $z_j^{\text{sel}} \leftarrow z_j^{(n_j^*)}$ 
11    Set  $H'_j = \min\{H', B_{\text{ctrl}} - t_j + 1\}$  if a budget is enforced, and  $H'_j = H'$  otherwise
12    Define  $\tilde{A}_j^+ = \tilde{A}_{j,1:H'_j}^{(n_j^*)}$ , execute  $A_j^{\text{env}} = \mathcal{N}_a^{-1}(\tilde{A}_j^+)$ , and observe  $y_{t_j+H'_j}$ 
13    Pad  $\tilde{A}_j^+$  to  $\tilde{A}_{j,\text{pad}}^+$  with mask  $M_j^+$ 
14    Compute  $S_{\text{exec}}(j)$  from  $C_j, M_j^C, \tilde{A}_{j,\text{pad}}^+, M_j^+$ , and  $y_{t_j+H'_j}$ 
15    Form  $s_j = (S_{\text{plan}}(j, n_j^*), S_{\text{vq}}(j, n_j^*), S_{\text{exec}}(j))$ 
16    Normalize  $s_j$ , compute  $r_j$ , update  $U_{j+1}$ , and set  $d_j = \mathbf{1}\{U_{j+1} > h_{\text{traj}}\}$ 
17    if  $d_j = 1$  then
18      └ Raise an alarm and stop
19    Update  $h_{j+1}$  by appending  $z_j^{\text{sel}}$  and retaining the most recent  $L_h$  tokens
20  Set  $t_{j+1} \leftarrow t_j + H'_j$  and  $j \leftarrow j + 1$ 
```

523 **EMA codebook update.** Let $\mathcal{V}_r = \{u_j : z_j = r\}$ be minibatch encoder outputs assigned to code
524 r . With EMA rate λ_{EMA} , counts and sums are updated as $N_r \leftarrow \lambda_{\text{EMA}} N_r + (1 - \lambda_{\text{EMA}})|\mathcal{V}_r|$ and
525 $m_r \leftarrow \lambda_{\text{EMA}} m_r + (1 - \lambda_{\text{EMA}}) \sum_{u \in \mathcal{V}_r} u$, followed by $e_r \leftarrow m_r / (N_r + \epsilon_{\text{EMA}})$. The encoder uses
526 a straight-through estimator and the commitment term in Eq. (15). Since centroids are updated by
527 EMA, no separate codebook regression loss is used.

528 **Dead-code restart and token histories.** Codes whose EMA counts remain below a minimum
529 threshold for W_{dead} consecutive updates are restarted from recent high-residual encoder outputs.
530 Restarts are used only during tokenizer training; after freezing, the codebook remains fixed. The
531 prior is trained with teacher-forced token histories from successful demonstrations. At decision j , h_j
532 contains at most the previous L_h success tokens and is left-padded with z_{BOS} . During deployment
533 and policy-induced calibration, h_j contains selected tokens z_i^{sel} , so the prior state follows the deployed
534 closed-loop policy rather than the full candidate set.

535 **Policy-induced prior adaptation.** If verified successful policy-induced rollouts are available, the
536 prior may be adapted on selected-token histories with the tokenizer and feature encoder fixed. The
537 adapted prior is frozen before final calibration. Adaptation rollouts are kept separate from threshold-
538 calibration and evaluation rollouts. After adaptation, all ranking normalizers, score normalizers,
539 reference levels, and trajectory thresholds are re-estimated for the adapted prior.

540 A.3 Actor instantiation and diagnostic distillation

541 FLOWGUARD can be attached to any chunk-based actor that produces horizon- H candidate chunks.
542 Our default actor is a conditional flow-matching sampler over normalized chunks. For a successful

543 training chunk $\tilde{A}_j^*$, sample $\epsilon_0 \sim \mathcal{N}(0, I)$ with the same shape as $\tilde{A}_j^*$ and $\lambda_{\text{fm}} \sim \mathcal{U}[0, 1]$. Define
 544 $X_{\lambda_{\text{fm}}} = (1 - \lambda_{\text{fm}})\epsilon_0 + \lambda_{\text{fm}}\tilde{A}_j^*$ and $\dot{X}_{\lambda_{\text{fm}}} = \tilde{A}_j^* - \epsilon_0$. The masked flow-matching loss is

$$\mathcal{L}_{\text{FM}}(\tilde{A}_j^*) = \mathbb{E} \left[\frac{\left\| M_j^A \odot (v_\varphi(X_{\lambda_{\text{fm}}}, \lambda_{\text{fm}}, C_j, M_j^C, M_j^A, h_j) - \dot{X}_{\lambda_{\text{fm}}}) \right\|_2^2}{d_a \max\{\sum_{h=1}^H M_{j,h}^A, 1\}} \right]. \quad (16)$$

545 The actor and success prior condition on the same context and bounded selected-token history. This
 546 shared conditioning supports candidate generation and timing-admissible scoring, but the monitoring
 547 claim does not rely on best-of- N improving task success.

548 For closed-loop diagnostics, we also evaluate a Flow-DP-Distill actor. It distills a diffusion-policy
 549 teacher on the same successful contexts using $\mathcal{L}_{\text{actor}} = \mathcal{L}_{\text{FM}}(\tilde{A}_j^*) + \lambda_{\text{DP}} \mathcal{L}_{\text{FM}}(\tilde{A}_{j,\text{DP}})$, where $\tilde{A}_{j,\text{DP}}$
 550 is the horizon-aligned normalized teacher chunk and $\lambda_{\text{DP}} \geq 0$. The teacher is queried only on training
 551 or adaptation contexts, never on threshold-calibration or evaluation rollouts. After distillation, the
 552 resulting actor is treated as the deployed actor: candidate samples, selected tokens, score distributions,
 553 reference levels, and trajectory thresholds are recomputed for the final actor and ranking rule.

554 A.4 Ranking-score normalization

555 The default ranking channel set is $\mathcal{Q}_{\text{rank}} = \{\text{plan}, \text{vq}\}$. The feedback-dependent score S_{exec} is
 556 excluded because it is unavailable before execution. Equation (8) uses normalized ranking scores.
 557 When ranking normalization is disabled, we set $\tilde{S}_{\chi}^{\text{rank}}(j, n) = S_{\chi}(j, n)$. Otherwise, we estimate a
 558 frozen ranking normalizer from successful training or adaptation contexts before detector calibration.
 559 For $\chi \in \mathcal{Q}_{\text{rank}}$, let m_{χ}^{rank} and a_{χ}^{rank} be the median and interquartile range of candidate scores. We use
 560 $\tilde{S}_{\chi}^{\text{rank}}(j, n) = (S_{\chi}(j, n) - m_{\chi}^{\text{rank}}) / \max\{a_{\chi}^{\text{rank}}, \epsilon_{\text{rank}}\}$. This normalization affects only candidate
 561 selection. Detection is calibrated on selected-chunk scores from the frozen ranking rule.

562 A.5 Execution-consistency model

563 The execution model predicts the feedback feature after the selected prefix is executed. It conditions
 564 on the pre-execution context and executed normalized prefix, not on the unexecuted suffix or raw
 565 environment-space actions A_j^{env} . During demonstration training, the demonstrated prefix is used;
 566 during deployment, the selected prefix is used. To retain a fixed horizon- H interface, define
 567 $\tilde{A}_{j,\text{pad},h}^+ = \tilde{A}_{j,h}^{(n_j^*)}$ for $h \leq H'_j$ and 0 otherwise, with $M_{j,h}^+ = \mathbf{1}\{h \leq H'_j\}$.
 568 Let $f_j^+ = \phi_o(y_{t_j+H'_j})$. The likelihood in Eq. (9) is parameterized as a diagonal Gaussian mixture:

$$p_{\xi}(f_j^+ | C_j, M_j^C, \tilde{A}_{j,\text{pad}}^+, M_j^+) = \sum_{m=1}^{M_{\text{exec}}} \lambda_{j,m}^{\text{mix}} \mathcal{N}(f_j^+; \mu_{j,m}, \text{diag}(\sigma_{j,m}^2)), \quad (17)$$

569 where $\lambda_{j,m}^{\text{mix}} \geq 0$ and $\sum_{m=1}^{M_{\text{exec}}} \lambda_{j,m}^{\text{mix}} = 1$ are mixture weights predicted together with $\mu_{j,m}$ and $\sigma_{j,m}$.
 570 The mixture parameters are predicted from a context embedding and a masked executed-prefix
 571 embedding. In multi-task implementations, the predictor may use a task embedding or task-specific
 572 heads; calibration remains task-specific. The model is trained on successful transitions by minimizing
 573 the same dimension-normalized negative log-likelihood used to define S_{exec} . The feedback map ϕ_o
 574 and execution model are frozen before calibration.

575 A.6 Success-only calibration protocol

576 Calibration follows the rollout split in Section 4.5. For task k , let $\mathcal{R}_k^{\text{norm}}$, $\mathcal{R}_k^{\text{ref}}$, and $\mathcal{R}_k^{\text{traj}}$ denote the
 577 task-specific splits for score normalization, reference estimation, and trajectory-threshold calibration.
 578 The hyperparameters τ_{shrink} , $\sigma_{\text{cal,min}}$, α_{ref} , ν , and ϵ_{rank} are fixed a priori and are never tuned on
 579 $\mathcal{R}_k^{\text{norm}}$, $\mathcal{R}_k^{\text{ref}}$, or $\mathcal{R}_k^{\text{traj}}$. Unless stated otherwise, the detection set is $\mathcal{Q}_{\text{det}} = \{\text{plan}, \text{vq}, \text{exec}\}$ as in
 580 Eq. (10). Changing the channel set or weights defines a new detector and requires recomputing all
 581 calibration quantities.

582 **Normalization samples.** For task k , progress bin ℓ , and channel $\chi \in \mathcal{Q}_{\text{det}}$, let $\mathcal{X}_{\ell,\chi}^{\text{norm},k}$ be
 583 valid scores $s_{j,\chi}^{(\rho)}$ with $b_j = \ell$ from $\rho \in \mathcal{R}_k^{\text{norm}}$. Empty bins use the pooled task-channel statistic.
 584 With $n_{\ell,\chi} = |\mathcal{X}_{\ell,\chi}^{\text{norm},k}|$ and $\lambda_{\ell,\chi} = n_{\ell,\chi}/(n_{\ell,\chi} + \tau_{\text{shrink}})$, let $(\bar{\mu}_{\ell,\chi}, \bar{\sigma}_{\ell,\chi})$ be bin statistics and
 585 $(\bar{\mu}_{\text{pool},\chi}, \bar{\sigma}_{\text{pool},\chi})$ pooled statistics. We use $\hat{\mu}_{\ell,\chi} = \lambda_{\ell,\chi}\bar{\mu}_{\ell,\chi} + (1 - \lambda_{\ell,\chi})\bar{\mu}_{\text{pool},\chi}$ and $\hat{\sigma}_{\ell,\chi} =$
 586 $\max\{\lambda_{\ell,\chi}\bar{\sigma}_{\ell,\chi} + (1 - \lambda_{\ell,\chi})\bar{\sigma}_{\text{pool},\chi}, \sigma_{\text{cal},\min}\}$.

587 **Reference levels.** Reference levels are estimated from $\mathcal{R}^{\text{ref}}$. For task k and bin ℓ , collect
 588 $\mathcal{S}_{\ell}^{\text{ref},k} = \{r_j^{(\rho)} : b_j = \ell, \rho \in \mathcal{R}_k^{\text{ref}}\}$. If nonempty, set $\kappa_{\ell}^{(k)}$ to the order statistic with index
 589 $\iota_{\ell,k}^{\text{ref}} = \min\{|\mathcal{S}_{\ell}^{\text{ref},k}|, \lceil (|\mathcal{S}_{\ell}^{\text{ref},k}| + 1)(1 - \alpha_{\text{ref}}) \rceil\}$. Empty or sparse bins use neighboring-bin or
 590 task-level pooled fallback. These reference levels center the CUSUM statistic and are not per-step
 591 conformal thresholds.

592 **Trajectory-level threshold.** For each rollout $\rho \in \mathcal{R}_k^{\text{traj}}$, compute $V^{(\rho)} = \max_j U_{j+1}^{(\rho)}$ using
 593 Eq. (12). Sort the task-specific maxima as $V_{(1)}^{(k)} \leq \dots \leq V_{(M_{\text{traj}}^{(k)})}^{(k)}$, where $M_{\text{traj}}^{(k)} = |\mathcal{R}_k^{\text{traj}}|$. For target
 594 false-alarm rate α_{FA} , define $\iota_{\text{traj}}^{(k)} = \lceil (M_{\text{traj}}^{(k)} + 1)(1 - \alpha_{\text{FA}}) \rceil$. If $\iota_{\text{traj}}^{(k)} \leq M_{\text{traj}}^{(k)}$, set $h_{\text{traj}}^{(k)} = V_{(\iota_{\text{traj}}^{(k)})}^{(k)}$;
 595 otherwise set $h_{\text{traj}}^{(k)} = +\infty$. Thus a finite split-conformal threshold requires $\alpha_{\text{FA}} \geq 1/(M_{\text{traj}}^{(k)} + 1)$.
 596 When the threshold is infinite, the detector is conservative and raises no finite alarm for that task. The
 597 strict alarm test $U_{j+1} > h_{\text{traj}}^{(k)}$ gives conservative tie handling.

598 **Offline proxy and policy-induced calibration.** When verified successful policy-induced rollouts
 599 are unavailable, successful demonstrations are used as an offline proxy. The demonstrated chunk is
 600 treated as selected: S_{plan} and S_{vq} are evaluated on that chunk, and S_{exec} is evaluated using feedback
 601 after the demonstrated prefix. This proxy preserves the scoring convention but does not model
 602 actor-induced closed-loop drift. When verified successful policy-induced rollouts are available, they
 603 are preferred because they match the deployed actor, candidate sampler, ranking rule, execution
 604 horizon, feature encoder, and scoring pipeline. If the actor, sampler, feature encoder, execution
 605 horizon, ranking rule, or any scoring component changes, the calibration statistics, reference levels,
 606 and trajectory thresholds are re-estimated for the resulting frozen pipeline.

607 **Adaptive deployment calibration.** The default detector uses the fixed threshold computed from
 608 $\mathcal{R}^{\text{traj}}$. If an adaptive conformal wrapper is enabled, it is applied only after the frozen detector has been
 609 calibrated. The adaptive update set contains only deployment rollouts whose success can be verified
 610 independently of the alarm decision. Completed successful rollouts are eligible. Alarm-aborted
 611 rollouts are eligible only when a simulator state, post hoc continuation or replay, or external verifier
 612 establishes that the corresponding unmonitored rollout would count as successful; failed or unverified
 613 rollouts are never used.

614 Let $h_m = h_{\text{traj}} + \delta_m$ be the operating threshold for the m -th verified successful deployment rollout,
 615 with $\delta_1 = 0$. After a verified successful rollout, define $e_m = 1$ if an alarm occurred and $e_m = 0$
 616 otherwise, then update

$$\delta_{m+1} = \delta_m + \eta_{\text{ACI}}(e_m - \alpha_{\text{FA}}), \quad (18)$$

617 optionally clipping δ_m to a pre-specified interval.

B Additional experimental results

This appendix reports additional diagnostics complementing the main text. Unless stated otherwise, low-dimensional experiments use the success-only training, calibration, and evaluation protocol in Section 5. Score normalization, reference levels, and trajectory thresholds are recomputed whenever the actor, sampler, feature encoder, execution horizon, ranking rule, score set, or calibration rule changes. When reported, S_{exec} -only is a FlowGuard ablation that isolates post-execution consistency.

Metric convention. For fixed, non-adaptive severity scores evaluated on a fixed rollout set, we report AUROC, balanced accuracy, TPR, and FPR, with FPR measured on successful rollouts. For interventional closed-loop monitoring, alarms may truncate execution and change the rollout distribution under each calibration rule; ACI also changes thresholds online. We therefore do not report AUROC for ACI and for rule-dependent interventional comparisons, and instead report thresholded alarm statistics with rule-specific outcome counts. Throughout, “–” denotes an undefined or inapplicable quantity.

B.1 Additional low-dimensional detection results

Table 3 reports per-task results for the default low-dimensional detector. AUROC is high on most tasks, showing strong trajectory-level separability between successful and corrupted rollouts. Thresholded metrics are more calibration-sensitive. This is clearest on TRANSPORT, where heavy-tailed successful trajectory scores make the operating threshold unstable: evaluation successes can exceed the calibration upper tail even when AUROC remains strong.

Table 3: Low-dimensional offline detection over five seeds after task-specific trajectory-level calibration. These are fixed-set, non-adaptive evaluations, so AUROC is reported.

Task	BalAcc	AUROC	TPR	FPR
LIFT	0.759±0.009	0.783±0.062	59±8%	7±8%
CAN	0.835±0.070	0.980±0.018	71±19%	4±8%
SQUARE	0.785±0.045	0.949±0.024	60±12%	3±4%
TOOLHANG	0.905±0.099	0.993±0.010	90±20%	9±13%
TRANSPORT	0.744±0.190	0.928±0.029	96±4%	47±41%
Macro	0.806±0.083	0.927±0.029	75±12%	14±15%

B.2 Failure-construction validation

Action-side corruptions should affect both the commanded prefix and subsequent feedback observations. If corrupted actions are paired with the original successful observation sequence, the feedback remains success-like and artificially weakens the post-execution consistency score. Table 4 separates observation-side failures, action corruptions without re-execution, action corruptions propagated through the simulator, and action corruptions propagated by learned residual dynamics.

Table 4: Failure-construction validation. Values are AUROC mean ± std over three seeds. “No re-execution” corrupts actions but keeps the original successful observations. “Simulator re-execution” propagates corrupted actions through the simulator, and “residual re-simulation” propagates them through learned residual dynamics. Bold marks the stronger action-side propagation protocol per task.

Task	Obs. only	No re-exec.	Simulator re-exec.	Residual re-sim.	Default mixture
LIFT	0.996±0.006	0.473±0.186	0.494±0.204	0.641±0.154	0.819±0.078
CAN	0.985±0.025	0.659±0.086	0.810±0.053	0.975±0.023	0.980±0.024
SQUARE	0.999±0.001	0.612±0.088	0.904±0.028	0.918±0.045	0.959±0.023
TOOLHANG	1.000±0.000	0.668±0.033	0.998±0.004	0.985±0.026	0.993±0.013
TRANSPORT	0.967±0.013	0.509±0.016	0.810±0.033	0.801±0.092	0.884±0.046

Table 5: Trajectory-level calibration diagnostics with $N_{\text{cal}} = 80$. Values are mean $\pm$ std over five seeds.

Task	M_{traj}	h_{traj}	Clean FPR	Failed TPR	BalAcc
LIFT	24	8.94 $\pm$ 11.85	0.07 $\pm$ 0.06	0.98 $\pm$ 0.04	0.96 $\pm$ 0.04
CAN	24	43.09 $\pm$ 78.26	0.04 $\pm$ 0.04	0.85 $\pm$ 0.31	0.91 $\pm$ 0.16
SQUARE	24	10.94 $\pm$ 3.15	0.06 $\pm$ 0.02	1.00 $\pm$ 0.00	0.97 $\pm$ 0.01

Table 6: Calibration-budget sensitivity on LIFT. “Split finite” is the fraction of seeds for which split conformal returns a finite threshold.

N_{cal}	M_{traj}	Split finite	FPR-Split	FPR-CRC	FPR-Interp
5	2	0%	0.00	0.18 $\pm$ 0.30	0.18 $\pm$ 0.30
10	3	0%	0.00	0.24 $\pm$ 0.34	0.24 $\pm$ 0.34
15	5	0%	0.00	0.10 $\pm$ 0.17	0.10 $\pm$ 0.17
20	6	0%	0.00	0.04 $\pm$ 0.09	0.04 $\pm$ 0.09
30	9	100%	0.04 $\pm$ 0.09	0.04 $\pm$ 0.09	0.04 $\pm$ 0.09
40	12	100%	0.06 $\pm$ 0.09	0.16 $\pm$ 0.26	0.06 $\pm$ 0.09
80	24	100%	0.04 $\pm$ 0.09	0.02 $\pm$ 0.04	0.06 $\pm$ 0.09
160	48	100%	0.04 $\pm$ 0.09	0.00 $\pm$ 0.00	0.04 $\pm$ 0.09

The table gives two checks. First, action-side failures must be propagated into future observations: without re-execution, AUROC drops on every task. Second, the action-side evidence is not an artifact of the learned residual model. Simulator re-execution gives strong AUROC on SQUARE, TOOLHANG, and TRANSPORT, and clearly improves over no re-execution on CAN. Residual re-simulation remains a useful scalable diagnostic when simulator re-execution is unavailable. LIFT is a quasi-static edge case in which both propagation protocols induce only weak post-execution deviations.

B.3 Calibration-budget and threshold sensitivity

Trajectory-level calibration converts correlated per-cycle scores into rollout alarms. Table 5 shows that, with $N_{\text{cal}} = 80$, split-conformal calibration keeps empirical FPR below the nominal $\alpha_{\text{FA}} = 0.10$ on the three main low-dimensional tasks while retaining high TPR.

Table 6 reports the finite-sample threshold cliff on LIFT. With too few successful trajectory maxima, split conformal returns $h_{\text{traj}} = \infty$, suppressing all alarms. CRC and linearly interpolated quantiles remain finite in this regime and are used only as operating-point diagnostics. We keep split conformal as the default because it has the clearest conservative false-alarm interpretation under rollout-level exchangeability.

B.4 Pre-execution candidate selection

The feedback-dependent score S_{exec} is unavailable before execution and is excluded from all ranking rules. This section evaluates only timing-correct pre-execution scores. We also include S_{rec} , a reconstruction-residual ranker, and RND-pre, a pre-execution random-network-distillation novelty ranker.

Low-dimensional smoothness diagnostic. Because the low-dimensional flow actor rarely completes full tasks in the small-data setting, Table 7 reports action smoothness. Lower values indicate more coherent horizon- H chunks. The combined $S_{\text{plan}} + S_{\text{vq}}$ ranker selects smoother chunks than random sampling on all three tasks, supporting its use as a pre-execution compatibility signal. This diagnostic does not establish improved task success by itself.

Real-rollout ranking with a stronger actor. Table 8 repeats the test with the Flow-DP-Distill actor, for which closed-loop success is meaningful. $S_{\text{plan}} + S_{\text{vq}}$ does not reliably improve success and can underperform random selection. Thus, these pre-execution scores are best interpreted as timing-correct compatibility scores whose rollout-level utility depends on the actor distribution.

Table 7: Low-dimensional best-of- N smoothness at $N = 32$. Lower is better. All ranking rules are pre-execution; the non-deployable reference summarizes candidate-set behavior.

Rule	LIFT	CAN	SQUARE
Random	13.82±4.12	21.04±13.14	25.22±4.51
S_{plan}	10.79±8.45	13.94±5.63	17.44±5.03
S_{vq}	13.34±8.50	14.79±7.41	20.39±4.70
$S_{\text{plan}} + S_{\text{vq}}$	5.30±0.97	13.76±7.42	15.72±6.06
S_{rec}	21.81±21.55	8.95±3.44	9.90±3.25
RND-pre	14.37±7.13	10.62±4.11	20.14±7.00
Reference	5.93±3.63	6.20±3.01	10.65±3.90

Table 8: Closed-loop best-of- N success rate with the Flow-DP-Distill actor at $N = 32$. Values are mean $\pm$ std over three seeds and eight rollouts per seed. All rules are pre-execution; S_{exec} is excluded because it requires post-execution feedback.

Task	Random	S_{plan}	S_{vq}	$S_{\text{plan}} + S_{\text{vq}}$	S_{rec}	RND-pre
LIFT	41.7±11.8%	12.5±10.2%	29.2±23.6%	16.7±15.6%	16.7±11.8%	66.7±25.7%
CAN	25.0±0.0%	4.2±5.9%	0.0±0.0%	0.0±0.0%	8.3±5.9%	0.0±0.0%
SQUARE	0.0±0.0%	0.0±0.0%	0.0±0.0%	0.0±0.0%	0.0±0.0%	0.0±0.0%

672 B.5 Visual encoder diagnostics

673 Table 9 evaluates frozen visual representations on LIFT, CAN, and SQUARE. These are offline
674 fixed-set evaluations, so AUROC is reported. Same-task DP-init features give the strongest offline
675 detection performance, while ImageNet-frozen features are weaker. This supports task-aligned visual
676 representations for success-only monitoring.

677 B.6 Visual monitoring and actor-induced calibration

678 Visual closed-loop monitoring introduces actor-induced shift: successful policy rollouts can have
679 score statistics different from successful demonstrations. It is also interventional, because alarms
680 truncate execution and alter the rollout distribution under each calibration rule. Outcome counts are
681 therefore rule-specific and should not be interpreted as fixed-rollout-set detector comparisons.

682 **Settings.** Tables 11 and 12 use seeds $\{42, 43, 44\}$ and eight evaluation rollouts per seed, for 24
683 rollouts per backbone–task–rule row. For each backbone–task setting, we train a separate Flow policy
684 on the corresponding visual feature backbone. Shared settings are $H = 16$, $H' = 8$, $N_{\text{cand}} = 4$,
685 $\mathcal{Q}_{\text{rank}} = \{\text{plan}, \text{vq}\}$, $\mathcal{Q}_{\text{det}} = \{\text{plan}, \text{vq}, \text{exec}\}$, and $\alpha_{\text{FA}} = 0.10$. For ACI, $\eta_{\text{ACI}} = 0.05$. All 24
686 scheduled rollouts per row are included in the reported counts.

687 **Rule-specific closed-loop counts.** Table 11 reports three mutually exclusive outcomes. “Completed
688 success” reaches the environment success condition without an alarm. “Task failure” fails before any
689 alarm. “Alarm-aborted” is censored by the monitor before the unmonitored task outcome is observed.
690 The counts therefore measure deployment behavior, not only detector thresholds.

691 **Thresholded alarm control.** Table 12 reports FAR on verified-success policy-induced rollouts.
692 The numerator is the number of alarms on verified-success rollouts; the denominator is the number of
693 verified-success rollouts. Alarm-aborted rollouts are included in this verified-success set only when
694 success is verified independently of monitor termination; otherwise they are treated as unverified and
695 excluded from the verified-success FAR calculation. This is a thresholded deployment diagnostic, not
696 AUROC. AUROC is not reported for ACI because the threshold adapts online and the closed-loop
697 rollout distribution is rule-dependent.

698 Tables 11 and 12 support the deployment-matched calibration claim. On LIFT, demo-static calibration
699 over-fires under both visual backbones: all 24 rollouts are alarm-aborted and verified-success FAR is
700 1.000. Actor-induced calibration with ACI reduces alarm-aborted rollouts to 3/24 for both backbones
701 and lowers FAR to 0.056 for BC-RNN features and 0.136 for DP-init features, close to $\alpha_{\text{FA}} = 0.10$.

Table 9: Visual offline detection with frozen encoders on LIFT, CAN, and SQUARE. Values are mean $\pm$ std over five seeds. DP-init uses the same-task DP-image encoder.

Encoder ϕ_ω	BalAcc	AUROC	TPR	FPR
Random-init	0.834 $\pm$ 0.086	0.939 $\pm$ 0.070	81 $\pm$ 16%	14 $\pm$ 22%
ImageNet-frozen	0.710 $\pm$ 0.088	0.688 $\pm$ 0.115	51 $\pm$ 19%	9 $\pm$ 11%
BC-RNN feat.	0.729 $\pm$ 0.150	0.865 $\pm$ 0.091	49 $\pm$ 33%	3 $\pm$ 4%
DP-init feat.	0.887$\pm$0.145	0.992$\pm$0.020	82$\pm$30%	4 $\pm$ 8%

Table 10: Compact visual closed-loop deployment summary. Except for the final column, entries are counts out of 24 rollouts. “No-monitor success” measures actor success without alarm termination. “Demo” and “Actor-static” report interventional alarm aborts. “ACI success/abort” gives completed successes and alarm aborts under actor-induced ACI calibration. “ACI alarm ratio” is alarms divided by verified-success rollouts; “–” indicates no verified-success rollouts.

Backbone	Task	No-mon. succ.	Demo abort	Actor-static abort	ACI succ./abort	ACI alarm ratio
BC-RNN feat.	LIFT	22/24	24/24	19/24	17/3	1/18
BC-RNN feat.	CAN	13/24	12/24	9/24	12/2	1/13
BC-RNN feat.	SQUARE	2/24	17/24	12/24	1/12	1/2
DP-init feat.	LIFT	21/24	24/24	17/24	19/3	3/22
DP-init feat.	CAN	9/24	15/24	18/24	6/14	2/8
DP-init feat.	SQUARE	1/24	18/24	22/24	0/14	–

On CAN, the benefit is backbone-dependent: ACI improves completed successes and reduces aborts for BC-RNN features, while DP-init features remain mixed. SQUARE is actor-bound, so its FAR estimates are sparse or undefined.

B.7 Calibration-design ablation

Table 13 replays the same low-dimensional train/calibration/evaluation splits under six calibration variants, changing only calibration and aggregation. Variants A–E all use successful calibration rollouts and a trajectory-level threshold; their FPR stays near the nominal $\alpha_{FA} = 0.10$, between 0.05 and 0.08. The full FLOWGUARD pipeline gives the lowest FPR among these calibrated variants while retaining comparable TPR. Variant F removes trajectory-level thresholding and alarms per cycle; its FPR rises to 0.894, confirming that repeated cycle-level tests inflate rollout-level false alarms. This supports progress-binned normalization, one-sided evidence, CUSUM accumulation, per-bin reference levels, and a final trajectory threshold.

B.8 No-monitor baselines and fixed-set calibration replay

No-monitor closed-loop baseline. Table 14 reports natural closed-loop actor performance when alarm-based termination is disabled. The monitor still computes per-cycle scores and trajectory statistics, but alarms cannot truncate execution; each rollout runs until environment termination or the time budget. This separates actor quality from monitor behavior. The actor is reliable on LIFT, weaker on CAN, and has very low success on SQUARE. Thus, the SQUARE closed-loop results in Table 11 are primarily actor-bound. Conversely, the no-monitor LIFT rows illustrate the cost of over-firing: demo-static calibration aborts all 24 LIFT rollouts under both backbones, whereas no-monitor execution succeeds in 22/24 and 21/24 rollouts.

Fixed-set actor-induced calibration comparison. The interventional visual comparison in Table 11 regenerates rollouts under each calibration rule, so it mixes calibration shift with rollout-distribution shift from alarm truncation. Table 15 removes this confound by replaying demo-static, actor-induced static, and actor-induced adaptive calibration on the same untruncated no-monitor score traces from Table 14. FAR is measured on successful no-monitor traces and TPR on failed no-monitor traces; BalAcc is reported only when both sides are defined. On LIFT and CAN, demo-static calibration repeatedly over-fires on successful actor-induced traces, including FAR = 1.00 on LIFT for both backbones. Actor-induced static calibration reduces fixed-set LIFT FAR to 0.00, showing that the

Table 11: Rule-specific closed-loop outcome counts for visual monitoring. Rollouts are regenerated under each calibration rule because alarms truncate execution. Each row aggregates three seeds and eight rollouts per seed.

Backbone	Task	Calibration rule	Completed success	Task failure	Alarm-aborted	Total
BC-RNN feat.	LIFT	Demo-static	0	0	24	24
BC-RNN feat.	LIFT	Actor-induced static	2	3	19	24
BC-RNN feat.	LIFT	Actor-induced + ACI	17	4	3	24
BC-RNN feat.	CAN	Demo-static	8	4	12	24
BC-RNN feat.	CAN	Actor-induced static	7	8	9	24
BC-RNN feat.	CAN	Actor-induced + ACI	12	10	2	24
BC-RNN feat.	SQUARE	Demo-static	2	5	17	24
BC-RNN feat.	SQUARE	Actor-induced static	3	9	12	24
BC-RNN feat.	SQUARE	Actor-induced + ACI	1	11	12	24
DP-init feat.	LIFT	Demo-static	0	0	24	24
DP-init feat.	LIFT	Actor-induced static	6	1	17	24
DP-init feat.	LIFT	Actor-induced + ACI	19	2	3	24
DP-init feat.	CAN	Demo-static	8	1	15	24
DP-init feat.	CAN	Actor-induced static	1	5	18	24
DP-init feat.	CAN	Actor-induced + ACI	6	4	14	24
DP-init feat.	SQUARE	Demo-static	0	6	18	24
DP-init feat.	SQUARE	Actor-induced static	0	2	22	24
DP-init feat.	SQUARE	Actor-induced + ACI	0	10	14	24

FPR reduction in the interventional study reflects real calibration-shift correction rather than only rule-specific truncation. The adaptive rule matches the static rule in most defined fixed-set cells because the actor-calibration pool is small and the update is slow. As in the interventional ACI analysis, AUROC is not reported for adaptive rows. The SQUARE rows remain sparse because the no-monitor actor has almost no successful rollouts.

B.9 Robustness diagnostics

Channel-weight robustness. Table 16 varies the weights on $\{S_{\text{plan}}, S_{\text{vq}}, S_{\text{exec}}\}$. Increasing the execution-score weight improves the macro operating point, consistent with S_{exec} being the primary detection signal. The conclusions remain unchanged across the tested settings, and all weights are fixed globally rather than tuned per task.

Visual nuisance stress. Table 17 compares feature-space nuisance proxies with engine-side state jitter. Directly perturbing cached features yields high nuisance AUROC, indicating sensitivity to off-manifold feature shifts. In contrast, engine-side jitter remains near chance because perturbations are propagated through the simulator before features are extracted. We therefore treat feature-space nuisance as a pessimistic stress test rather than a realistic rendered-nuisance benchmark.

B.10 Qualitative rollout diagnostics

Figure 3 shows representative closed-loop rollouts. The detected SQUARE failure has a late rise in S_{exec} that drives the CUSUM statistic above threshold. The undetected SQUARE failure has elevated scores, but accumulated evidence remains below the trajectory threshold. The clean LIFT success shows transient score fluctuations without an alarm. These examples illustrate that FLOWGUARD alarms on accumulated calibrated evidence rather than isolated score spikes.

Table 12: Visual closed-loop false-alarm rate under actor-induced shift. Entries report alarms on verified-success policy-induced rollouts divided by verified successful rollouts. Lower is better; “–” indicates undefined FAR because no verified-success rollouts were available.

Backbone	Task	Calibration rule	Alarms / verified succ.	FAR
BC-RNN feat.	LIFT	Demo-static	19/19	1.000
BC-RNN feat.	LIFT	Actor-induced static	16/18	0.889
BC-RNN feat.	LIFT	Actor-induced + ACI	1/18	0.056
BC-RNN feat.	CAN	Demo-static	3/11	0.273
BC-RNN feat.	CAN	Actor-induced static	2/9	0.222
BC-RNN feat.	CAN	Actor-induced + ACI	1/13	0.077
BC-RNN feat.	SQUARE	Demo-static	1/3	0.333
BC-RNN feat.	SQUARE	Actor-induced static	0/3	0.000
BC-RNN feat.	SQUARE	Actor-induced + ACI	1/2	0.500
DP-init feat.	LIFT	Demo-static	17/17	1.000
DP-init feat.	LIFT	Actor-induced static	16/22	0.727
DP-init feat.	LIFT	Actor-induced + ACI	3/22	0.136
DP-init feat.	CAN	Demo-static	2/10	0.200
DP-init feat.	CAN	Actor-induced static	8/9	0.889
DP-init feat.	CAN	Actor-induced + ACI	2/8	0.250
DP-init feat.	SQUARE	Demo-static	–	–
DP-init feat.	SQUARE	Actor-induced static	–	–
DP-init feat.	SQUARE	Actor-induced + ACI	–	–

Table 13: Calibration-design ablation. Values are mean $\pm$ std across three tasks and three seeds. Variants A–E use successful calibration rollouts and a trajectory-level threshold; variant F is a per-cycle diagnostic without trajectory-level thresholding.

Variant	Score statistic	Progress bins	CUSUM	κ_b ref.	Traj. thr.	FPR	TPR
A	Global max	no	no	–	yes	0.083 $\pm$ 0.030	0.417 $\pm$ 0.250
B	Progress max	yes	no	–	yes	0.083 $\pm$ 0.043	0.378 $\pm$ 0.288
C	Global CUSUM	no	yes	global ν	yes	0.061 $\pm$ 0.060	0.428 $\pm$ 0.262
D	Progress CUSUM, no κ_b	yes	yes	no	yes	0.072 $\pm$ 0.046	0.317 $\pm$ 0.299
E	Full FLOWGUARD	yes	yes	per-bin κ_b	yes	0.050$\pm$0.045	0.339 $\pm$ 0.291
F	Per-cycle diagnostic	yes	no	–	no	0.894 $\pm$ 0.094	0.994$\pm$0.014

Table 14: No-monitor closed-loop outcomes. Alarm-based termination is disabled, so $N_{\text{abort}} = 0$ by construction. Each row aggregates three seeds and eight rollouts per seed.

Backbone	Task	N_{succ}	N_{fail}	N_{abort}	N_{total}	Success
BC-RNN feat.	LIFT	22	2	0	24	91.7%
BC-RNN feat.	CAN	13	11	0	24	54.2%
BC-RNN feat.	SQUARE	2	22	0	24	8.3%
DP-init feat.	LIFT	21	3	0	24	87.5%
DP-init feat.	CAN	9	15	0	24	37.5%
DP-init feat.	SQUARE	1	23	0	24	4.2%

Table 15: Fixed-set actor-induced calibration comparison. All rules are replayed on the same untruncated no-monitor traces, removing rollout-distribution shift from alarm truncation. “Actor-static” denotes actor-induced static calibration, and “Actor+ACI” denotes actor-induced adaptive calibration. AUROC is not reported for adaptive rows; other “–” entries indicate undefined metrics due to no verified successful evaluation rollouts.

Backbone	Task	Rule	FAR	TPR	BalAcc	AUROC
BC-RNN feat.	LIFT	Demo-static	1.00	1.00	0.50	0.50
BC-RNN feat.	LIFT	Actor-static	0.00	0.50	0.75	0.55
BC-RNN feat.	LIFT	Actor+ACI	0.00	0.50	0.75	–
BC-RNN feat.	CAN	Demo-static	1.00	1.00	0.50	0.39
BC-RNN feat.	CAN	Actor-static	0.57	0.36	0.40	0.35
BC-RNN feat.	CAN	Actor+ACI	0.57	0.36	0.40	–
BC-RNN feat.	SQUARE	Demo-static	0.00	0.86	0.93	0.91
BC-RNN feat.	SQUARE	Actor-static	1.00	0.77	0.39	0.64
BC-RNN feat.	SQUARE	Actor+ACI	1.00	0.77	0.39	–
DP-init feat.	LIFT	Demo-static	1.00	1.00	0.50	0.82
DP-init feat.	LIFT	Actor-static	0.00	0.00	0.50	0.73
DP-init feat.	LIFT	Actor+ACI	0.00	0.00	0.50	–
DP-init feat.	CAN	Demo-static	1.00	0.93	0.47	0.40
DP-init feat.	CAN	Actor-static	0.00	0.67	0.83	0.89
DP-init feat.	CAN	Actor+ACI	0.00	0.67	0.83	–
DP-init feat.	SQUARE	Demo-static	–	1.00	–	–
DP-init feat.	SQUARE	Actor-static	–	0.78	–	–
DP-init feat.	SQUARE	Actor+ACI	–	0.78	–	–

Table 16: Macro robustness to score-channel weights over five tasks.

Weights ($w_{\text{plan}}, w_{\text{vq}}, w_{\text{exec}}$)	BalAcc	AUROC	TPR	FPR
(1/3, 1/3, 1/3)	0.806	0.917	71%	10%
(0.25, 0.25, 0.50)	0.828	0.925	74%	8%
(0.10, 0.10, 0.80)	0.854	0.927	78%	7%

Table 17: Engine-side versus feature-space nuisance stress. Values are nuisance AUROC; lower is better.

Task	Engine-side state jitter		Feature-space proxy	
	$\sigma = 0.005 \downarrow$	$\sigma = 0.02 \downarrow$	Object pose $\downarrow$	Gripper pose $\downarrow$
LIFT	0.380±0.062	0.490±0.062	0.886±0.038	0.933±0.088
CAN	0.513±0.095	0.440±0.142	0.930±0.032	0.988±0.018
SQUARE	0.477±0.087	0.623±0.090	0.929±0.017	1.000±0.001

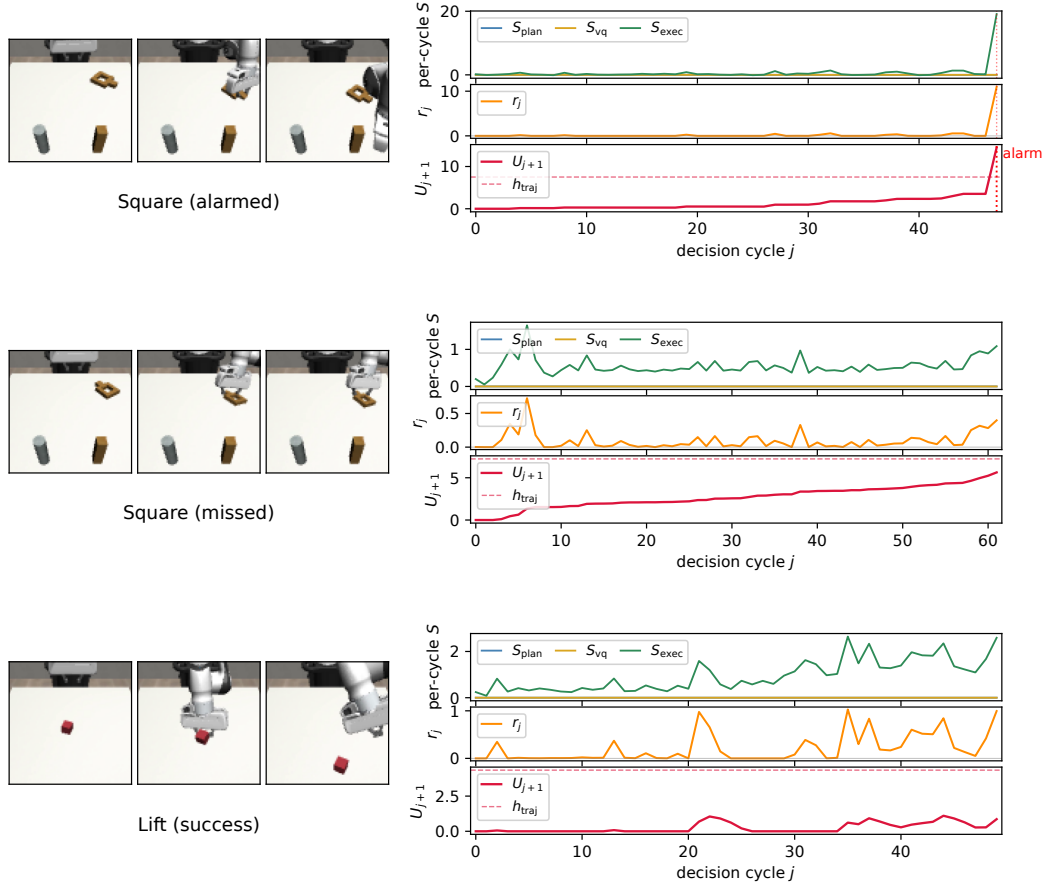


Figure 3: **Qualitative closed-loop diagnostics.** Each row shows visual frames, per-cycle score channels S_{plan} , S_{vq} , and S_{exec} , the one-sided aggregate score r_j , and the post-cycle CUSUM statistic U_{j+1} with trajectory threshold $h_{\text{traj}}^{(k)}$.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

Justification: Yes. The claims in the abstract and introduction are generally well aligned with the paper's contributions.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Yes. The paper discusses limitations in the conclusion section.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Yes. The paper states the assumptions for the theoretical result and provides a proof sketch and justification in the main text.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Yes. The paper discloses the experimental setup, datasets, splits, model components, calibration protocol, metrics, and implementation details needed to reproduce the main experimental results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: No. The code and implementation details are not publicly released at the time of submission, but we plan to release them upon acceptance.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Yes. The paper specifies the datasets, splits, model settings, calibration protocol, and hyperparameters to understand the experimental results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Yes. The paper reports mean and standard deviation across multiple seeds for the main experimental results where applicable.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: No. The paper does not provide detailed information about compute resources or execution time.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: Yes. The research conducted in the paper conforms to the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: No. The paper does not include a discussion of broader societal impacts.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: Not applicable.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Yes. The paper properly credits the datasets, models, and prior work used, and follows their respective terms of use.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: Not applicable.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: Not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: Not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.

- 1063
- 1064
- 1065
- 1066
- 1067
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
 - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

1068 **16. Declaration of LLM usage**

1069 Question: Does the paper describe the usage of LLMs if it is an important, original, or
1070 non-standard component of the core methods in this research? Note that if the LLM is used
1071 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
1072 scientific rigor, or originality of the research, declaration is not required.

1073 Answer: [N/A]

1074 Justification: Not applicable.

1075 Guidelines:

- 1076
- 1077
- 1078
- 1079
- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
 - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.